



RAPPORT DE SYNTHÈSE

TaskFlow

Gestionnaire de tâches conteneurisé et déployé

Projet	Déployer et exploiter une application web
Nom du projet	TaskFlow
Groupe	Groupe : _____
Membres	Simão Silva · _____ · _____
URL publique	https://taskflow.planbadge.fr
Dépôt	github.com/simaosilva85/taskflow
Date	Juin 2026

Seuls les étudiants mentionnés dans ce rapport et visibles dans les commits du dépôt sont considérés comme contributeurs au projet.

— Sommaire

1. Présentation du projet
2. Dépôt GitHub
3. Architecture
4. Déploiement
5. Exploitation
6. Sauvegardes — la règle 3-2-1
7. Support utilisateur
8. Retour d'expérience
9. Auto-évaluation au regard de la grille

En une phrase. TaskFlow est une application web de gestion de tâches, développée en Node.js, empaquetée avec Docker et déployée publiquement via Coolify ; ce rapport documente son cycle de vie complet, du code à l'exploitation en production.

1. Présentation du projet

Objectif de l'application

TaskFlow permet à un utilisateur de **gérer sa liste de tâches** : ajouter une tâche avec un niveau de priorité, la marquer comme faite ou à faire, et la supprimer. Les données sont **persistées** en base, donc conservées entre les sessions et les redéploiements.

L'objectif pédagogique n'est pas la complexité fonctionnelle, mais de **démontrer la maîtrise du cycle de vie complet** d'une application : code → conteneur → déploiement → supervision → exploitation.

Public visé

Tout utilisateur ayant besoin d'un gestionnaire de tâches simple et rapide, accessible depuis un navigateur, sans installation. L'interface est en français, responsive (utilisable sur mobile) et sans courbe d'apprentissage.

Principales fonctionnalités

- Créer une tâche avec une priorité (**basse, normale, haute**) ;
- Marquer une tâche comme faite / à faire ;
- Supprimer une tâche ;
- Filtrer l'affichage (toutes / à faire / faites) et voir le nombre de tâches ;
- Indicateur de disponibilité du service en direct (sonde `/health`).

Solution d'intelligence artificielle

Aucune IA dans l'application TaskFlow est une application classique (CRUD) : elle n'embarque pas de modèle d'IA. Des outils d'IA ont uniquement servi à *assister le développement* (génération de code, rédaction), conformément à ce qu'autorise l'énoncé.

Pile technique

Couche	Technologie	Rôle
Frontend	HTML / CSS / JavaScript (vanilla)	Interface utilisateur
Backend / API	Node.js 20 + Express	API REST <code>/api/tasks</code>
Base de données	PostgreSQL 16	Persistence des tâches
Conteneurisation	Docker + Docker Compose	Empaquetage et orchestration
Déploiement	Coolify	Mise en ligne, HTTPS, supervision

2. Dépôt GitHub

Le code source est hébergé dans un dépôt **public** :

`https://github.com/simaosilva85/taskflow`

Contributions du groupe

Chaque membre commite sous sa propre identité Git. L'historique montre une progression par étapes claires, avec des messages « métier » suivant la convention `type: description` (voir `CONTRIBUTING.md`).

Type de commit	Exemple de message
<code>feat</code>	API REST des tâches avec persistance PostgreSQL
<code>feat</code>	interface web de gestion des tâches (filtres, état du service)
<code>chore</code>	conteneurise l'application (Dockerfile + docker-compose)
<code>chore</code>	scripts de sauvegarde et restauration (règle 3-2-1)
<code>docs</code>	README et guide de contribution du groupe

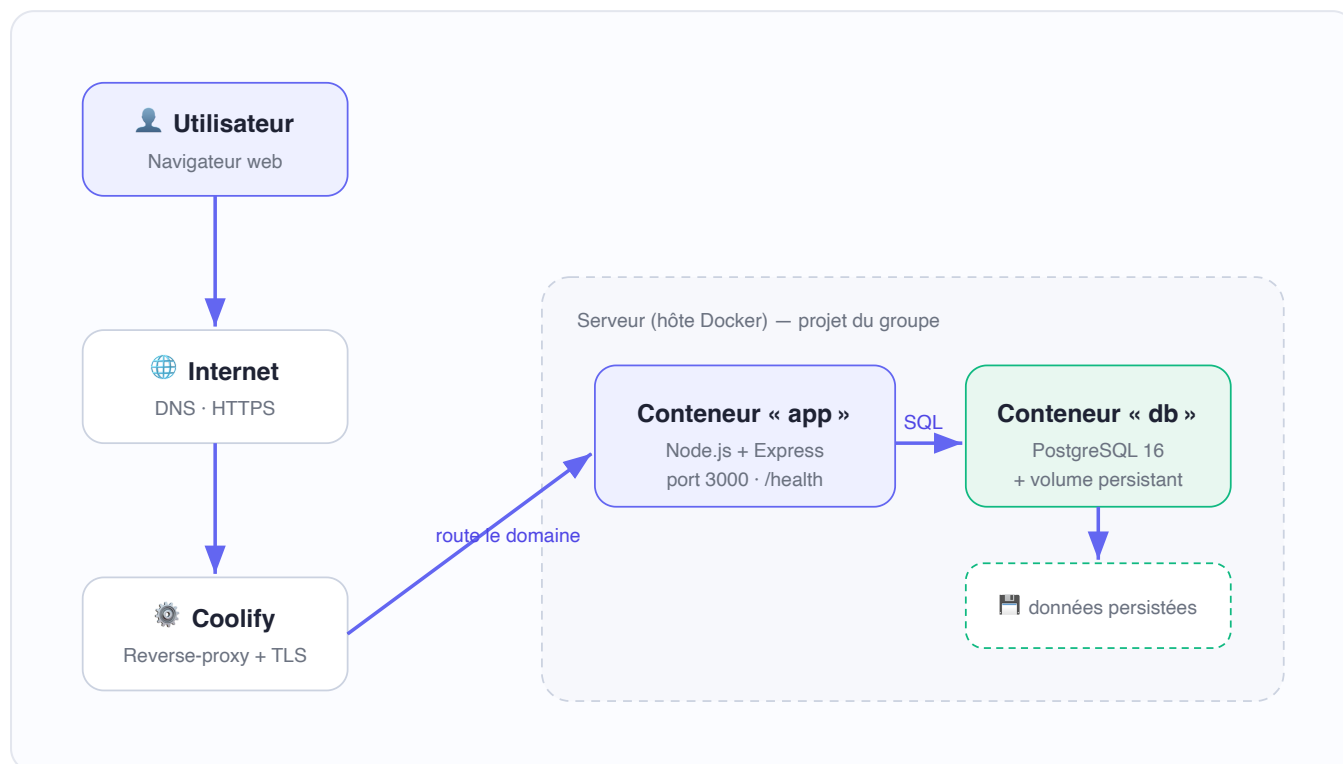
Correspondance pseudo GitHub ↔ étudiant

À compléter si les pseudos ne permettent pas d'identifier directement les étudiants (exigence de l'énoncé).

Pseudo GitHub	Étudiant·e	Contributions principales
simaosilva85	Simão Silva	_____
_____	_____	_____
_____	_____	_____

3. Architecture

L'architecture retenue sépare le serveur web de sa base de données, le tout exposé sur Internet par Coolify qui assure le reverse-proxy et le HTTPS.



Description des briques

Brique	Responsabilité
Conteneur app	Sert le frontend, expose l'API REST, journalise et publie <code>/health</code> .
Conteneur db	PostgreSQL ; stocke les tâches dans un volume pour la persistance.
Coolify	Construit les images, route le domaine, gère le certificat HTTPS et redémarre les conteneurs.

Choix d'architecture. Séparer l'application de la base (deux conteneurs) reflète une vraie mise en production : on peut redéployer l'app sans toucher aux données, et sauvegarder la base indépendamment.

4. Déploiement

La chaîne de déploiement

```
VS Code → GitHub → Docker → Coolify → Internet
(code)   (source) (image)  (mise en  (URL
                               ligne+TLS)  publique)
```

Comment l'application a été déployée

- Le code est poussé sur la branche `main` du dépôt GitHub.
- Dans Coolify : **New Resource** → **Repository**, build pack **Docker Compose**. Coolify lit le `docker-compose.yml`, construit l'image `app` via le `Dockerfile` et démarre la base `db`.
- Les secrets (`PGUSER`, `PGPASSWORD`, `PGDATABASE`) sont définis dans l'onglet *Environment* de Coolify — **jamais** dans le dépôt.
- Un **domaine** `https://taskflow.planbadge.fr` est associé ; Coolify génère automatiquement le certificat **HTTPS** (Let's Encrypt).
- Le port interne exposé est `3000`.

Technologies utilisées

Node.js 20, Express, PostgreSQL 16, Docker, Docker Compose, Coolify, GitHub.

Principales difficultés rencontrées

Difficulté	Solution apportée
L'application démarrait avant que PostgreSQL soit prêt.	Ajout d'une attente avec ré-essais (<code>waitForDb</code>) et d'un <code>healthcheck</code> sur la base dans Compose (<code>depends_on: condition: service_healthy</code>).
Ne pas exposer les mots de passe dans le dépôt.	Variables d'environnement définies côté Coolify, fichier <code>.env.example</code> fourni comme modèle, <code>.env</code> ignoré par Git.
Savoir si l'application est réellement disponible.	Endpoint <code>/health</code> qui vérifie aussi la base, branché sur le health check de Docker et de Coolify.

5. Exploitation

Supervision

Trois niveaux de surveillance, du plus interne au plus externe :

- **Endpoint applicatif** `/health` — vérifie que le serveur et la base répondent. Renvoie `200 {status:"ok"}` ou `503 {status:"degraded"}`, avec la version déployée.
- **Health check Docker / Coolify** — intégré au `Dockerfile` et au `docker-compose.yml` ; Coolify affiche l'état et peut redémarrer un conteneur défaillant.
- **Uptime Kuma** (déjà déployé en TP) — sonde `https://taskflow.planbadge.fr/health` chaque minute et **alerte** (mail / Discord) en cas d'indisponibilité.

Logs

L'application écrit ses logs sur la sortie standard : événements applicatifs au format **JSON** (démarrage, connexion à la base, erreurs) et accès HTTP au format `combined`. Ils sont consultables dans l'onglet **Logs** de Coolify, ou en local via `docker compose logs -f app`.

```
{"ts":"2026-06-18T12:00:01.244Z","level":"info","message":"TaskFlow démarré","port":3000}
::1 - - [18/Jun/2026:12:00:05] "POST /api/tasks HTTP/1.1" 201 86
```

Utilité : tracer l'activité, diagnostiquer une erreur, suivre les démarrages/arrêts et aider le support à comprendre un incident.

Stratégie de mise à jour

- Développement sur une branche, *pull request* relue par un autre membre.
- Fusion dans `main` → Coolify reconstruit l'image et redéploie.
- `restart: unless-stopped` garantit le redémarrage automatique des conteneurs.
- La version exposée par `/health` permet de confirmer la mise à jour.

Stratégie de sauvegarde

Détaillée à la section suivante (règle 3-2-1). Les données vivent dans un volume PostgreSQL et sont exportées par `scripts/backup.sh`.

6. Sauvegardes — la règle 3-2-1

Protéger les données métier (les tâches) suppose de pouvoir les restaurer après une panne, une erreur humaine ou une suppression accidentelle. Nous appliquons la règle de référence **3-2-1** :

3 copies

- la base en production (volume) ;
- un dump quotidien local ;
- une copie distante.

2 supports différents

- volume du serveur ;
- stockage objet (S3) ou disque externe.

1 copie hors site

- sur un autre hébergeur / cloud, à l'abri d'un incident du serveur principal.

Restauration testée

- une sauvegarde n'a de valeur que si la restauration fonctionne :

`scripts/restore.sh` sert à le vérifier.

Mise en œuvre concrète

```
# Dump quotidien compressé + rotation (7 derniers), planifiable via cron
./scripts/backup.sh
# → backups/taskflow_2026-06-18_02-00-00.sql.gz

# Restauration (testée) depuis une sauvegarde
./scripts/restore.sh backups/taskflow_2026-06-18_02-00-00.sql.gz
```

La copie hors site s'obtient en envoyant le fichier `.sql.gz` vers un stockage objet distant après chaque dump.

7. Support utilisateur

Si un utilisateur signale un problème (« je ne peux pas ajouter de tâche », « le site ne répond pas »), nous suivons une démarche structurée :

1. **Recueillir l'information** : que faisait l'utilisateur, à quelle heure, quel message d'erreur, quel navigateur.
2. **Vérifier la disponibilité** : consulter `/health` et le tableau de bord Uptime Kuma — le service est-il en ligne ?
3. **Consulter les logs** dans Coolify autour de l'heure de l'incident pour retrouver la requête fautive (code 4xx/5xx) ou une erreur de base.
4. **Reproduire** le problème de notre côté pour le confirmer.
5. **Corriger et communiquer** : appliquer un correctif (nouveau déploiement) puis informer l'utilisateur du rétablissement.

Atout pour le support. Comme chaque réponse HTTP et chaque erreur sont journalisées, le support peut relier le témoignage d'un utilisateur à une trace précise dans les logs, et la version déployée est lisible dans `/health`.

8. Retour d'expérience

✅ Ce qui a bien fonctionné

- Docker Compose : l'app et la base démarrent ensemble avec une seule commande.
- Le déploiement depuis GitHub vers Coolify, rapide une fois le dépôt en place.
- L'endpoint `/health` : simple à brancher sur Docker et Uptime Kuma.
- L'HTTPS automatique fourni par Coolify.

⚠️ Ce qui a été difficile

- Gérer l'ordre de démarrage app / base (résolu par les health checks).
- Comprendre où définir les variables d'environnement côté Coolify.
- Garder les secrets hors du dépôt.

Ce que nous améliorerions avec plus de temps

- Authentification des utilisateurs (chaque personne ses propres tâches) ;
- Tests automatisés exécutés en intégration continue (GitHub Actions) ;
- Sauvegarde hors site entièrement automatisée vers un stockage objet ;
- Tableau de bord de métriques (temps de réponse, nombre de requêtes).

9. Auto-évaluation au regard de la grille

Récapitulatif de la couverture des critères d'évaluation du projet.

Critère	Points	Comment c'est couvert
Dépôt GitHub	3	Dépôt public, historique par étapes, messages métier, guide de contribution, correspondance pseudo/étudiant.
Dockerisation	4	Dockerfile (non-root, healthcheck, cache optimisé) + docker-compose (app + base + volume).
Déploiement Coolify	4	Déploiement depuis GitHub, URL publique HTTPS, variables d'environnement, port 3000.
Monitoring et exploitation	3	<code>/health</code> applicatif, health check Docker/Coolify, Uptime Kuma, logs structurés exploitables.
Réflexion sur les sauvegardes	2	Règle 3-2-1 expliquée et mise en œuvre (scripts de dump/rotation et de restauration testée).
Qualité du rapport	4	Document structuré, schéma d'architecture, sections complètes, mise en forme soignée.
Total visé	20	Cycle de vie complet couvert et documenté.

Annexe — commandes utiles

```
docker compose up --build      # démarrer l'application
docker compose logs -f app     # consulter les logs
curl https://taskflow.planbadge.fr/health # vérifier la disponibilité
./scripts/backup.sh            # sauvegarder la base
```